

Deep Dive into `useReducer` and `useMemo`

Understanding useReducer

An alternative to `useState` for managing complex state logic. It centralizes state transitions into a single **reducer function**, making state updates more predictable and easier to test.

```
const [state, dispatch] = useReducer(reducer, initialArg, init?);
```

reducer

A pure function `(state, action) ⇒ newState` that calculates the next state based on the current state and the dispatched action.

dispatch

A function used to send actions to the reducer. It triggers a re-render with the new state returned by the reducer.

initialArg

The initial value used to calculate the state. Can be a primitive, object, or array.

state

The current state value managed by the hook, updated whenever an action is dispatched.

useReducer in Action: Counter Example

```
const initialState = { count: 0 };

function reducer(state, action) {
  switch (action.type) {
    case 'increment':
      return { count: state.count + 1 };
    case 'decrement':
      return { count: state.count - 1 };
    default:
      throw new Error();
  }
}

function Counter() {
  const [state, dispatch] = useReducer(reducer, initialState);

  return (
    <button onClick={() => dispatch({ type: 'increment' })}>
      Count: {state.count}
    </button>
  );
}
```

1. The Reducer

A pure function that determines the next state. It never mutates the **state**; it always returns a new object.

2. The Action

An object (e.g., **{ type: 'increment' }**) that describes what happened. The reducer uses this to decide the update logic.

3. Dispatching

Calling **dispatch** sends the action to the reducer. React then re-renders the component with the returned state.

When to Choose `useReducer` over `useState`



Complex State Logic

When state involves multiple sub-values that change together or the next state depends on the previous one.

Centralized Logic

To move state update logic outside of the component, making it easier to test and maintain independently.



Performance Optimization

When passing `dispatch` down to deep components; its stable identity prevents unnecessary re-renders.

"useReducer is usually preferable to useState when you have complex state logic that involves multiple sub-values or when the next state depends on the previous one."

Understanding useMemo

A React Hook that lets you **cache the result of a calculation** between re-renders. It implements **memoization**, ensuring that expensive functions are only re-executed when their specific dependencies change.

```
const cachedValue = useMemo(() => calculateValue(a, b), [a, b]);
```

calculateValue

A pure function that takes no arguments and returns the value you want to cache. React calls it during the initial render and subsequent renders only if dependencies change.

cachedValue

The result of the calculation. On re-renders, React returns the stored value if dependencies are unchanged, otherwise it re-calculates.

dependencies

An array of reactive values (props, state, variables) referenced inside the calculation. React compares them using **Object.is**.

Performance

Primarily used to skip expensive recalculations and to maintain referential equality for objects or arrays passed as props to optimized components.

useMemo in Action: Filtered List

In this example, `useMemo` ensures that the expensive filter operation only runs when `items` or `query` changes.

Prevents re-filtering on every keystroke or parent re-render if the data remains the same.

```
function ItemList({ items }) {
  const [query, setQuery] = useState('');

  // Memoize the filtered result
  const filteredItems = useMemo(() => {
    console.log('Filtering ... ');
    return items.filter(item =>
      item.name.includes(query)
    );
  }, [items, query]); // Dependencies

  return (
    <div>
      <input
        value={query}
        onChange={e => setQuery(e.target.value)}
      />
      <ul>
        {filteredItems.map(item => (
          <li key={item.id}>{item.name}</li>
        ))}
      </ul>
    </div>
  );
}
```

When to Use useMemo



Expensive Calculations

When a function performs computationally intensive tasks like heavy data transformations or complex filtering that shouldn't re-run on every render.



Referential Equality

To ensure objects or arrays passed as props maintain the same reference, preventing unnecessary re-renders of child components wrapped in `React.memo`.



Dependency Stability

When a value is a dependency for other hooks like `useEffect` or `useCallback`, memoizing it prevents those hooks from triggering prematurely.

OPTIMIZATION CAUTION

Overusing `useMemo` adds memory overhead and complexity. Always profile your application using React DevTools to identify actual bottlenecks before optimizing.

useState vs. useReducer Comparison

Feature	useState	useReducer
Simplicity	Best for basic state (primitives, simple objects).	More boilerplate; better for complex logic.
Update Logic	Direct updates via setter functions.	Decoupled logic via actions and reducers.
State Structure	Independent state variables.	Interdependent sub-values in one object.
Scalability	Harder to manage as complexity grows.	Highly scalable; predictable transitions.
Performance	Standard React re-render behavior.	Stable dispatch identity prevents re-renders.

useMemo vs. useCallback Comparison

Feature	useMemo	useCallback
Purpose	Memoizes a computed value (result of a function).	Memoizes a function instance itself.
Returns	The cached result of the function.	The cached function itself.
Primary Use Case	Expensive calculations or maintaining referential equality for objects/arrays.	Preventing unnecessary re-renders of child components that receive callbacks as props.
Dependencies	Re-calculates the value if dependencies change.	Re-creates the function if dependencies change.

Note: `useCallback(fn, deps)` is equivalent to `useMemo(() => fn, deps)`.

Best Practices and Common Pitfalls

✔ Best Practices

Keep Reducers Pure

Reducers should only calculate the next state. Never perform side effects (API calls, logging) inside them.

Descriptive Action Types

Use clear, domain-specific strings for action types (e.g., 'ORDER_COMPLETED') to improve debugging.

Profile Before Optimizing

Use React DevTools to confirm a calculation is actually expensive before wrapping it in `useMemo`.

⚠ Common Pitfalls

Stale Dependency Arrays

Missing variables in dependency arrays lead to stale closures and bugs that are hard to track.

Over-Memoization

Memoizing cheap operations adds more overhead (memory/comparison) than it saves in execution time.

Direct State Mutation

Mutating state objects instead of returning new ones prevents React from detecting changes and re-rendering.

Conclusion and References

Mastering **useReducer** and **useMemo** allows you to build React applications that are both architecturally sound and highly performant.

Choose **predictability** for state and **efficiency** for computation.

OFFICIAL DOCUMENTATION

[↗ useReducer API Reference](#)

[↗ useMemo API Reference](#)

DEEP DIVES

[📖 LogRocket: Ultimate useReducer Guide](#)

[📖 Refine: useMemo Guide with Examples](#)

[📖 Josh W. Comeau: Understanding useMemo](#)